Contents

SQL Joins Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to explain and choose Oracle SQL joins before final SQL is accepted.

The workflow forces Copilot to map table relationships, state row-preservation behavior, identify cardinality and duplicate risks, produce small generic row examples, and validate the final join logic.

Use this chain for queries where correctness depends on choosing between inner joins, left joins, full joins, cross joins, semi joins with `EXISTS`, or anti joins with `NOT EXISTS`.

Required Local Preparation

- Have the business question, expected output grain, and required row preservation behavior ready.
- Prepare compact schema context for relevant tables, primary keys, foreign keys, natural keys, nullable join columns, filter columns, and known one-to-one, one-to-many, or many-to-many relationships.
- Gather representative sample rows or row-count expectations for the tables being joined.
- Keep validation evidence available, such as pre-join row counts, post-join row counts, duplicate checks by output key, unmatched-row checks, or reconciliation totals.

Prompt Sequence

1. `01-relationship-intake.md`: collect the business objective, candidate tables, expected output grain, join keys, and missing relationship context.
2. `02-join-choice-explanation.md`: require Copilot to choose and explain join types with row-preservation mini-examples.

3. `03-duplicate-risk-check.md`: force cardinality, many-to-many, and duplicate risk checks before SQL is accepted.
4. `04-example-driven-query.md`: draft Oracle SQL from the approved join map and explain how each join keeps or drops rows.
5. `05-final-join-validation.md`: validate row preservation, duplicate checks, and cardinality assumptions before accepting final SQL.

Related Reference Drawer

- `references/oracle-sql/sections/join_patterns.md`

Use the drawer when detailed join guidance is needed, such as Oracle join syntax, join pattern selection, semi and anti join patterns, row-preservation rules, or duplicate-risk examples. Do not duplicate long SQL tutorial content from the drawer in prompt responses.

Relationship To Existing Flows

- Use `prompt-chains/sql-general/` first when the SQL request is broad, simple, or not clearly centered on join behavior.
- Use this chain after `sql-general` when the task depends on explaining table relationships, choosing join types, or proving row-preservation behavior.
- Use `prompt-chains/query-tuning/` when the primary problem is runtime, indexing, optimizer behavior, or `EXPLAIN PLAN` evidence.
- Use this chain before `query-tuning` when a slow join query must first be proven logically correct.

SQL Joins Chain - 01 Relationship Intake

Act as a senior Oracle SQL assistant focused on join correctness.

This is a join explanation and validation workflow, not a final SQL workflow yet. Do not produce final SQL until the relationship map, join choices, cardinality risks, duplicate checks, and row-preservation checks are complete.

Business objective:

```
<paste the reporting question, business rule, expected result, or acceptance criteria here>
```

Candidate tables and fields:

```
<paste table names, aliases, relevant columns, primary keys, foreign keys, natural keys, nullab
```

Expected output:

```
<paste expected output grain, required columns, rows that must be preserved, rows that may be e
```

Known sample rows or validation evidence:

```
<paste sample rows, table row counts, distinct key counts, duplicate checks, unmatched-row chec
```

Return:

1. Restated business objective.
2. Expected output grain and row-preservation requirement, if inferable.

3. Candidate left-side and right-side tables for each relationship.
4. Known join keys and missing join keys.
5. Known or likely cardinality for each relationship: one-to-one, one-to-many, many-to-one, or many-to-many.
6. Nullable keys, optional relationships, or filters that could affect row preservation.
7. Missing schema, key, sample-row, or validation context needed before join types can be accepted.
8. Specific clarification questions required before writing final SQL.

Point me to `references/oracle-sql/sections/join_patterns.md` if detailed join guidance is needed. Do not repeat long tutorial content from that drawer.

SQL Joins Chain - 02 Join Choice Explanation

Continue the Oracle SQL join explanation workflow.

Using the relationship intake and any clarifications already provided, choose the appropriate Oracle SQL join pattern for each table relationship. Do not produce final SQL yet.

Relationship intake and clarifications:

```
<paste Copilot's intake response and any answered clarification questions here>
```

Additional schema or business context:

```
<paste any new keys, row-count notes, sample rows, optional relationship rules, or required fi
```

Return one row per relationship using exactly this table shape:

```
left table | right table | join type | join keys | cardinality | preserved rows |
duplicate risk | validation check
```

For each relationship, explain:

1. Why the selected join type matches the business requirement.
2. Which rows are preserved and which rows can be dropped.
3. Whether the relationship is one-to-one, one-to-many, many-to-one, or many-to-many.
4. Whether the join can introduce duplicate output rows.
5. The validation check needed before the join can be accepted.

Cover these join patterns when relevant:

1. Inner join when only matching rows should remain.
2. Left join when every row from the left table must remain.
3. Full join when unmatched rows from both sides must remain.
4. Cross join only when every combination is intentional and bounded.
5. Semi join with `EXISTS` when the query needs to test whether a matching row exists without multiplying rows.
6. Anti join with `NOT EXISTS` when the query needs rows with no matching related row.

After the table, provide generic mini-examples for each join type used. Use small invented tables with two or three rows and explain which rows are kept, dropped, or duplicated. Keep the examples generic and short.

Do not approve final SQL until every relationship has a join type, join key, cardinality statement, preserved-row statement, duplicate-risk statement, and validation check.

Use `references/oracle-sql/sections/join_patterns.md` only when detailed join guidance is needed.

SQL Joins Chain - 03 Duplicate Risk Check

Continue the Oracle SQL join explanation workflow.

Before final SQL is accepted, test the completed join map for cardinality and duplicate risks.

Completed join map:

`<paste the join map table from step 02 here>`

Known row counts, key counts, or sample rows:

`<paste table row counts, distinct join-key counts, duplicate counts by key, sample rows, or kn`

Return:

1. Expected output grain.
2. Join relationships that appear one-to-one and why.
3. Join relationships that appear one-to-many or many-to-one and how that affects output rows.
4. Join relationships that may be many-to-many and why they are risky.
5. Relationships where nullable keys, non-unique keys, date-effective records, status filters, or missing predicates could multiply rows.
6. Semi join or anti join opportunities where `EXISTS` or `NOT EXISTS` would avoid unnecessary row multiplication.
7. Cross joins that must be removed or explicitly justified as intentional and bounded.
8. Exact validation checks to run before final SQL is accepted.

For validation checks, include Oracle SQL snippets or pseudocode for:

1. Counting rows before and after each join.
2. Counting duplicate output keys after each join.
3. Counting distinct join keys on both sides.
4. Finding unmatched rows for outer joins.
5. Detecting many-to-many joins before the final query.

If any relationship can multiply rows beyond the expected output grain, do not approve final SQL. State whether the next step is to add a missing predicate, pre-aggregate, deduplicate with a business rule, switch to `EXISTS`, switch to `NOT EXISTS`, or ask for more context.

Point me to `references/oracle-sql/sections/join_patterns.md` if detailed join guidance is needed, but do not repeat long tutorial content from that drawer.

SQL Joins Chain - 04 Example-Driven Query

Continue the Oracle SQL join explanation workflow.

Draft Oracle SQL only after the join map and duplicate-risk checks are complete. The SQL must preserve the approved output grain and row-preservation behavior.

Approved join map:

`<paste the completed join map table here>`

Duplicate and cardinality review:

`<paste Copilot's duplicate-risk findings and required mitigations here>`

Query requirements:

`<paste selected columns, filters, grouping, ordering, bind variables, and final output requiren`

Return:

1. The proposed Oracle SQL.
2. A short explanation of each join in query order.
3. For each join, a generic mini-example showing how rows are kept, dropped, preserved, or multiplied.
4. Any `EXISTS` semi join or `NOT EXISTS` anti join explanation, including why it avoids row multiplication.
5. Any cross join explanation, including why every combination is intentional and bounded.
6. Any duplicate-risk mitigation used in the SQL, such as pre-aggregation, deduplication by a business rule, or a more specific join predicate.
7. Validation queries or checks that must be run before accepting the SQL.

Do not hide uncertainty. If the approved join map is incomplete, or if the duplicate-risk review found unresolved one-to-many or many-to-many risks, stop and list the missing checks instead of producing final SQL.

Use `references/oracle-sql/sections/join_patterns.md` only when detailed join guidance is needed.

SQL Joins Chain - 05 Final Join Validation

Continue the Oracle SQL join explanation workflow.

Use this final checklist before accepting an Oracle SQL query whose correctness depends on join choices.

Final or proposed Oracle SQL:

```
<paste the final or proposed Oracle SQL here>
```

Completed join map:

`<paste the completed join map table here>`

Validation results:

`<paste row counts, duplicate checks, distinct key counts, unmatched-row checks, sample-output`

Return:

1. Whether the final SQL matches the completed join map.

2. Confirmation that every join has stated left table, right table, join type, join keys, cardinality, preserved rows, duplicate risk, and validation check.
3. Confirmation that inner joins drop only rows the business logic allows to be dropped.
4. Confirmation that left joins preserve all required left-side rows.
5. Confirmation that full joins preserve required unmatched rows from both sides.
6. Confirmation that cross joins are intentional, bounded, and not accidental.
7. Confirmation that `EXISTS` semi joins are used where existence checks should not multiply rows.
8. Confirmation that `NOT EXISTS` anti joins are used where missing related rows are required.
9. Confirmation that one-to-one, one-to-many, many-to-one, and many-to-many assumptions have validation evidence.
10. Confirmation that duplicate and cardinality checks were run before final SQL acceptance.
11. Remaining correctness risks, if any.
12. Whether the final SQL can be accepted as validated.

If any join is missing cardinality evidence, duplicate checks, row-preservation checks, or unresolved many-to-many risk handling, do not approve the final SQL. List the missing item and the next validation check to run.

Point me to `references/oracle-sql/sections/join_patterns.md` if detailed join guidance is needed, but do not repeat long tutorial content from that drawer.